pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Word Embeddings

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Words as vectors

Day 3: a column per term

"bond" and "bonds" are two unrelated columns; most of a row is zero.

Today: 100 numbers per word

Words used in similar contexts get rows that point the same way. Every number is filled.

44,558

GloVe words kept by the course

100

numbers per word

0

words with a capital letter

Real documents, described

An 8-K section's model output

Allowed names	Never called
the item the model assigns	a default signal
the model's probability or score in this file	a distress flag or a warning
the filer's item	a credit event or credit view

An indenture's terms

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

How a word vector works

The problem	A program compares numbers, and a word is not one
The vector	A list of 100 numbers learned from a large body of text: similar company, similar lists
No labels	One number means nothing alone; two lists compare
You control	Which table to load, and how a word is cut and lower-cased in a lookup
What comes out	A table with a row per word, 100 numbers wide, and a way to compare any two rows

A word as a vector

44,558

words, float32

100

numbers per word

0

capital letters: GloVe is uncased

```
vectors, words = textkit.load_word_vectors("course7_glove_6b_100d.npz")
position["bond"]                # 1,846: GloVe's frequency order
vectors[position["bond"]][:3]   # 0.8918, 0.3513, 0.1522: nothing on their own
np.linalg.norm(bond)            # 5.8566: compare directions, not lengths
```

Every lookup must lower-case its word first.

Where the vectors come from

The method	an unsupervised learning algorithm ... global word-word co-occurrence statistics
The file	Wikipedia 2014 + Gigaword 5 (6B tokens, 400K vocab, uncased ...): glove.6B.zip
The licence	Public Domain Dedication and License v1.0

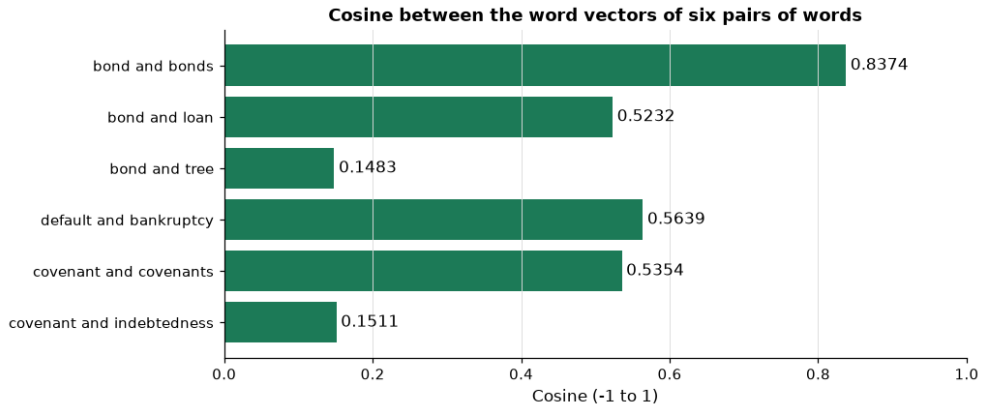
Pennington, Socher, Manning: EMNLP 2014, pages 1532 to 1543. word2vec named, not used.

Sources, read 8 October 2026: nlp.stanford.edu/projects/glove; aclanthology.org/D14-1162; arxiv.org/abs/1301.3781

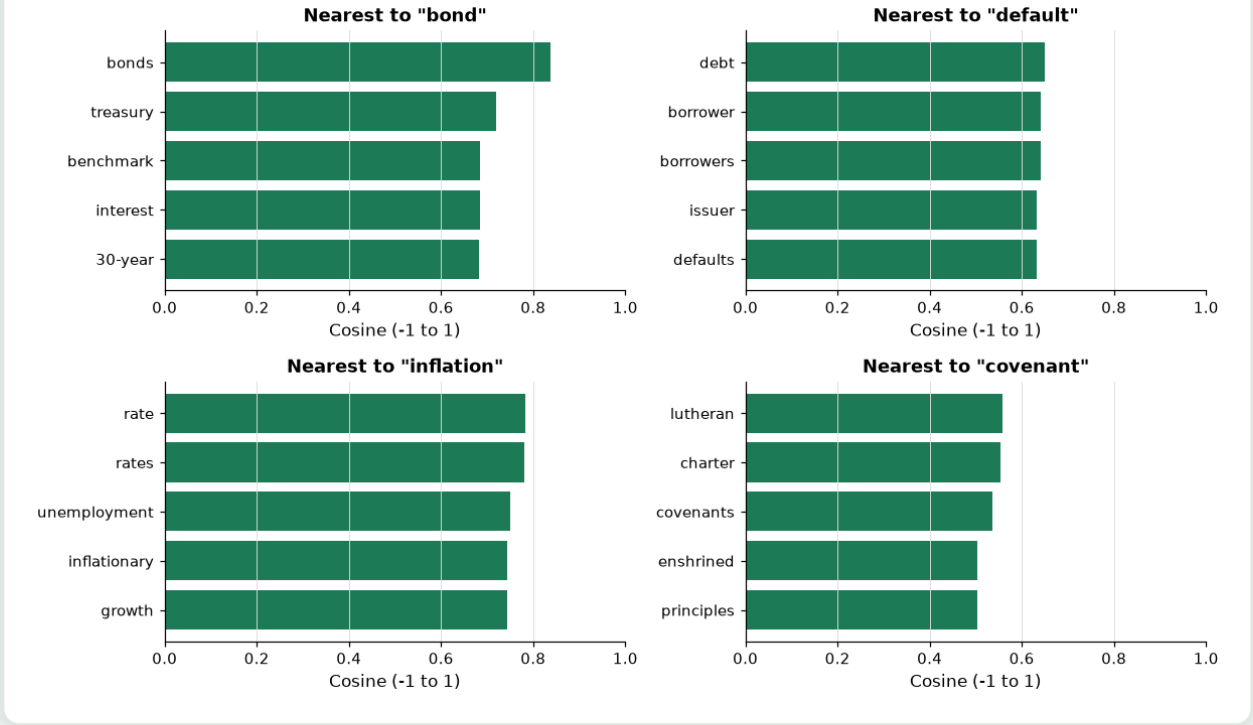
How cosine and nearest words work

The cosine	The dot product of two vectors divided by the product of their lengths
Directions	It compares directions and ignores lengths: 1 same, 0 unrelated, -1 opposite
Nearest words	The same, for one word against every row at once, then sorted
You control	The word and how many neighbours to return
What comes out	Words with cosines, largest first, the word itself left out, ties broken by the word

Cosine and nearest words

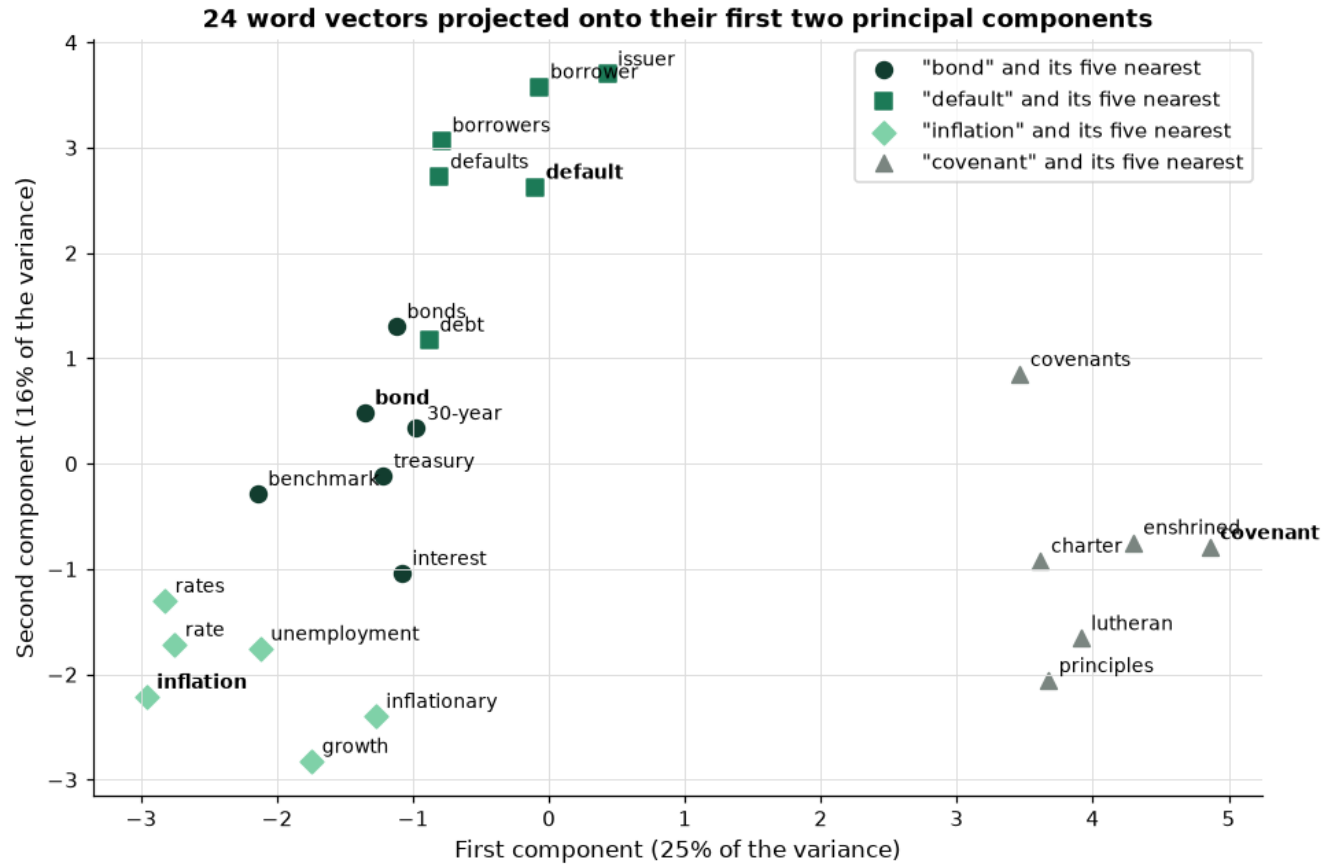


Word	Its three nearest words
bond	bonds, treasury, benchmark
default	debt, borrower, borrowers
inflation	rate, rates, unemployment
covenant	lutheran, charter, covenants



A nearest word is used in similar contexts in the training text. It is not a meaning.

Words in two dimensions



What the picture shows

The four query words and their five nearest words, 24 vectors, squeezed onto the two directions along which they differ most. Words used in the same company land in the same part of the picture. A rough guide: two axes keep only part of the variance.

What is missing

422

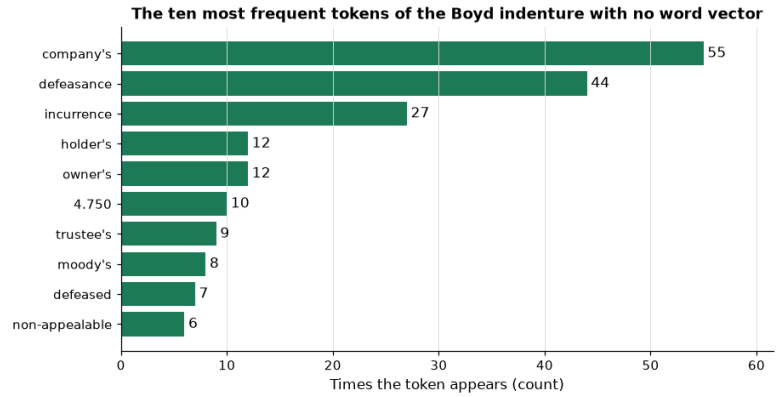
of 63,810 tokens have no vector

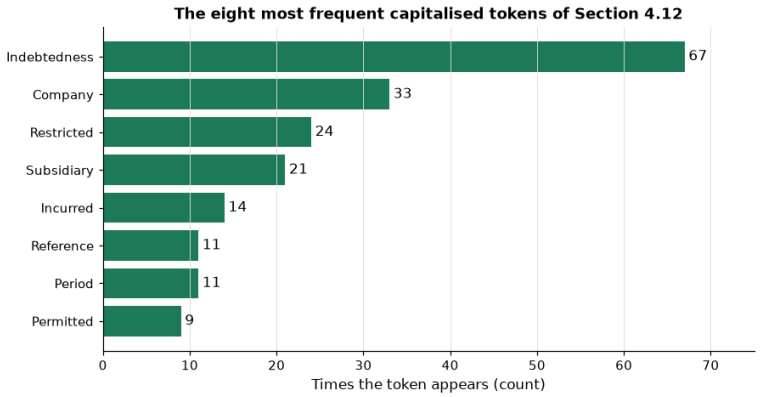
137

distinct missing tokens

374

capitalised, of 1,840 in 4.12





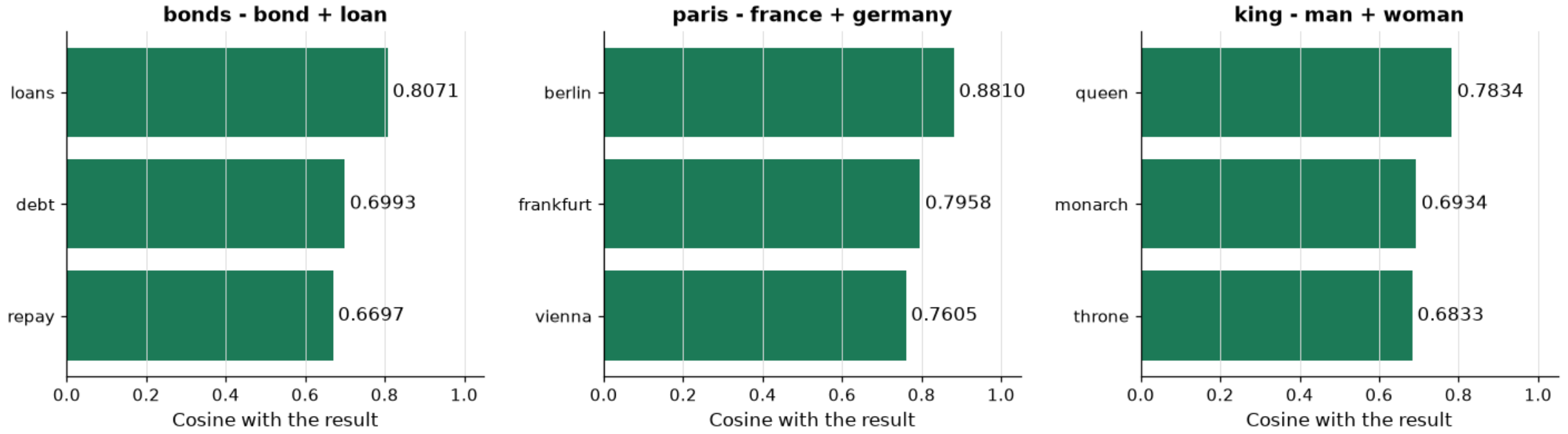
notes: note, letters, write; indebtedness: repayments, borrowings, debts; lien: soong, siew, hsieh

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 4.12.

How the analogy works

The idea	If a direction stands for a relation, adding one pair's difference to a third word lands near its partner
The steps	Vector of b minus a plus c; the rows with the largest cosine to it
Leave out	The three input words, or the answer would be one of them
You control	The three words and how many neighbours to return
What comes out	Nearest words, always, whether or not the relation holds

Arithmetic on vectors, once

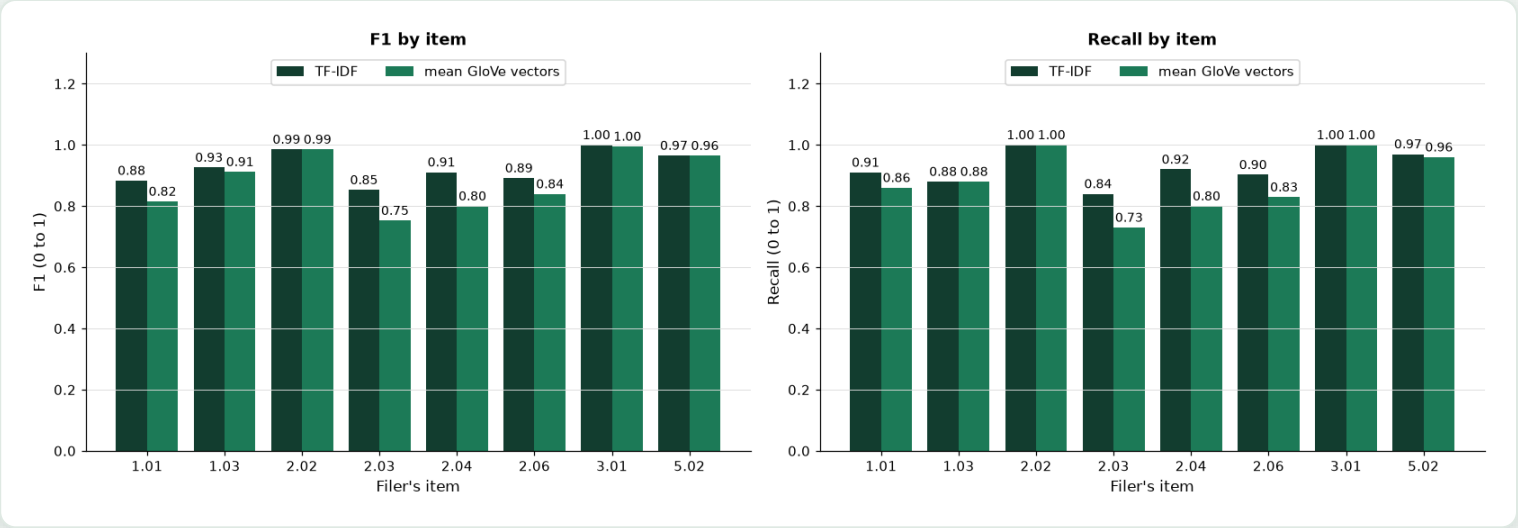


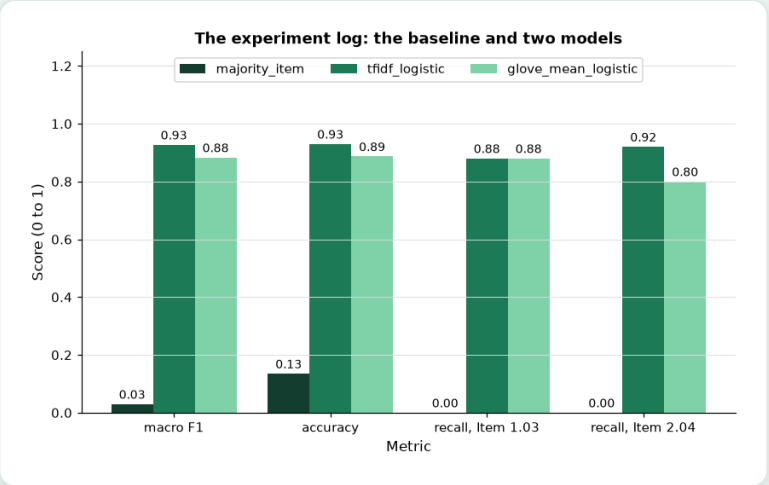
It always returns some word, right or wrong. A nearest word is not a fact.

How document vectors work

The problem	A classifier needs one row of numbers per section, and a section has many words
The mean	Look up each token's vector, skip the tokens with none, take the mean of the rest
One row	Every section is 100 numbers, whatever its length
You control	The vectors, which tokens count, and the classifier
What comes out	A row per section, and the share of its tokens that had no vector

Document vectors as features



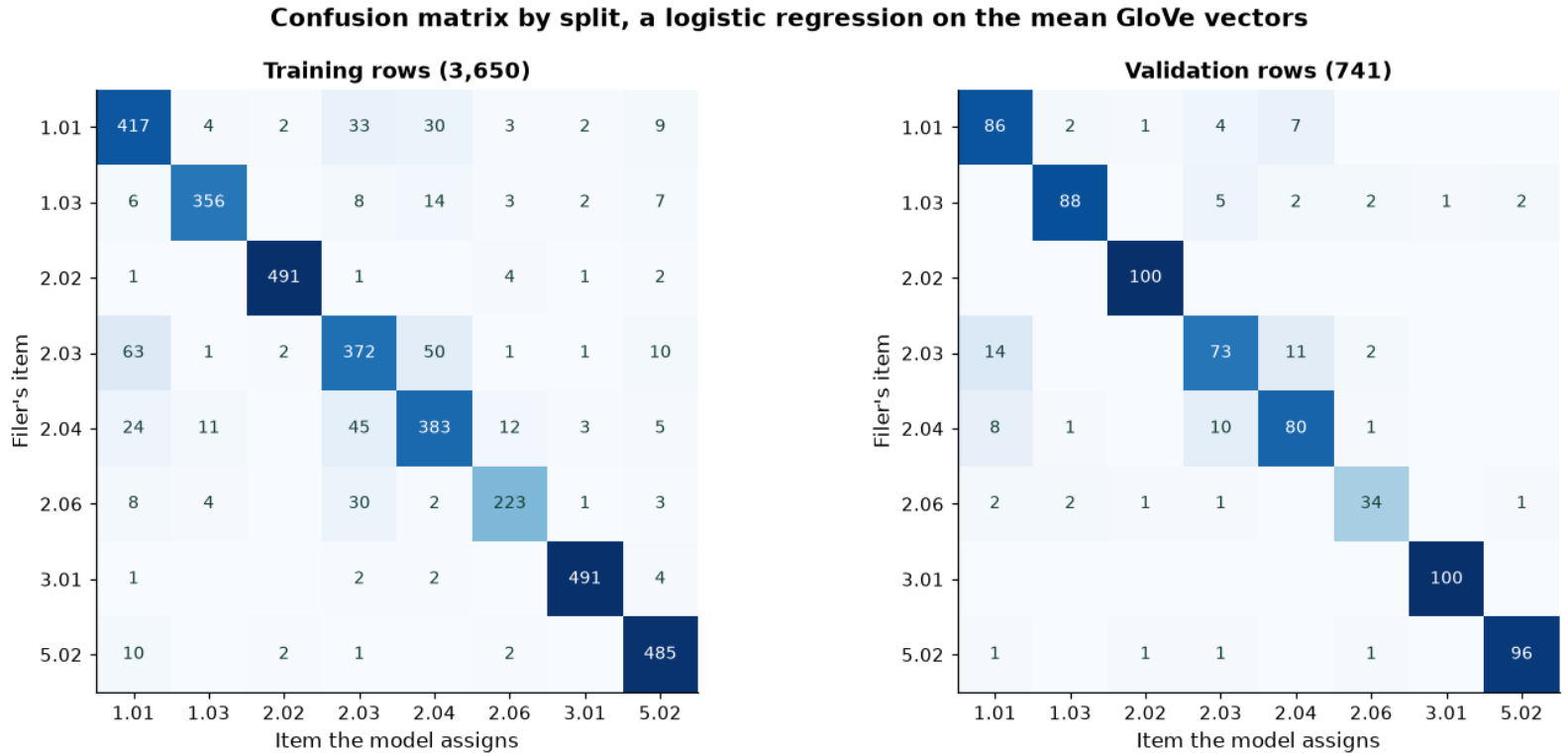


Accuracy: majority item 0.1350, TF-IDF 0.9298, GloVe mean 0.8866.

Fitted on 3,650 sections of 2019 to 2023, scored on 741 of 2024 (validation); 735 of 2025 closed.

Macro F1 0.9269 against 0.8830: the item the model assigns, in this file.

The mean-vector model by split



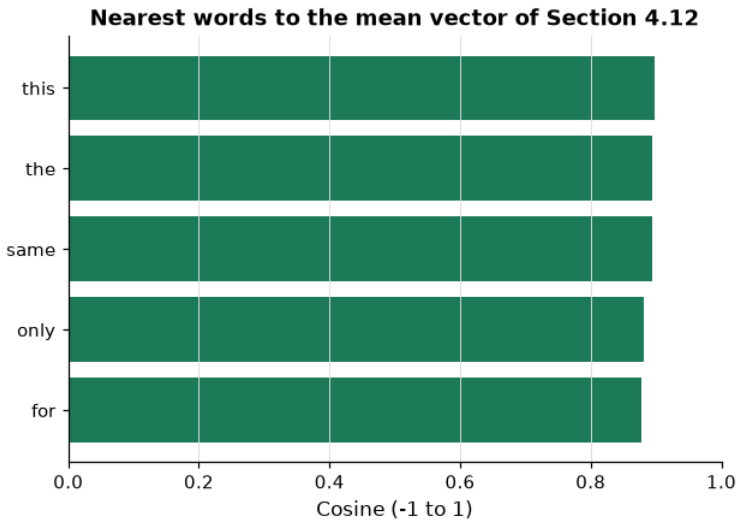
Off the diagonal

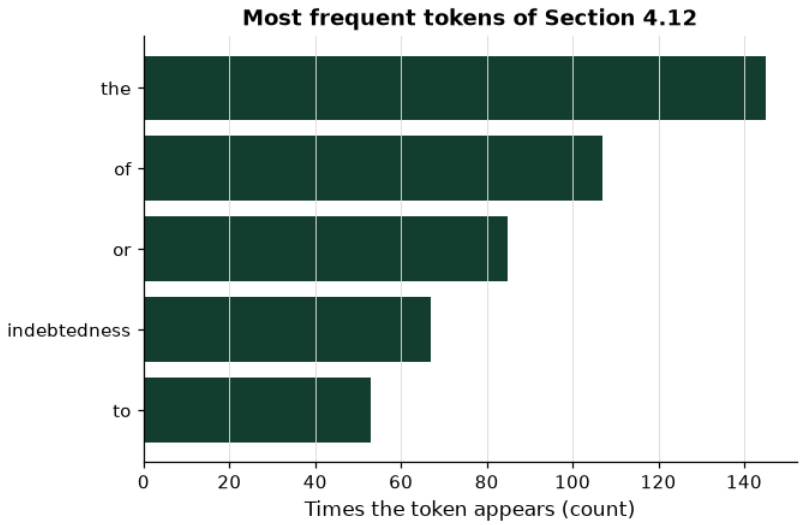
Training: 432 of 3,650 sections. Validation: 84 of 741. The right panel is the one to read; the test rows are closed until day 10.

Rows: the filer's item. Columns: the item the model assigns.

What averaging loses

```
"the trustee may remove the company"
"the company may remove the trustee"   # identical vectors: True
```





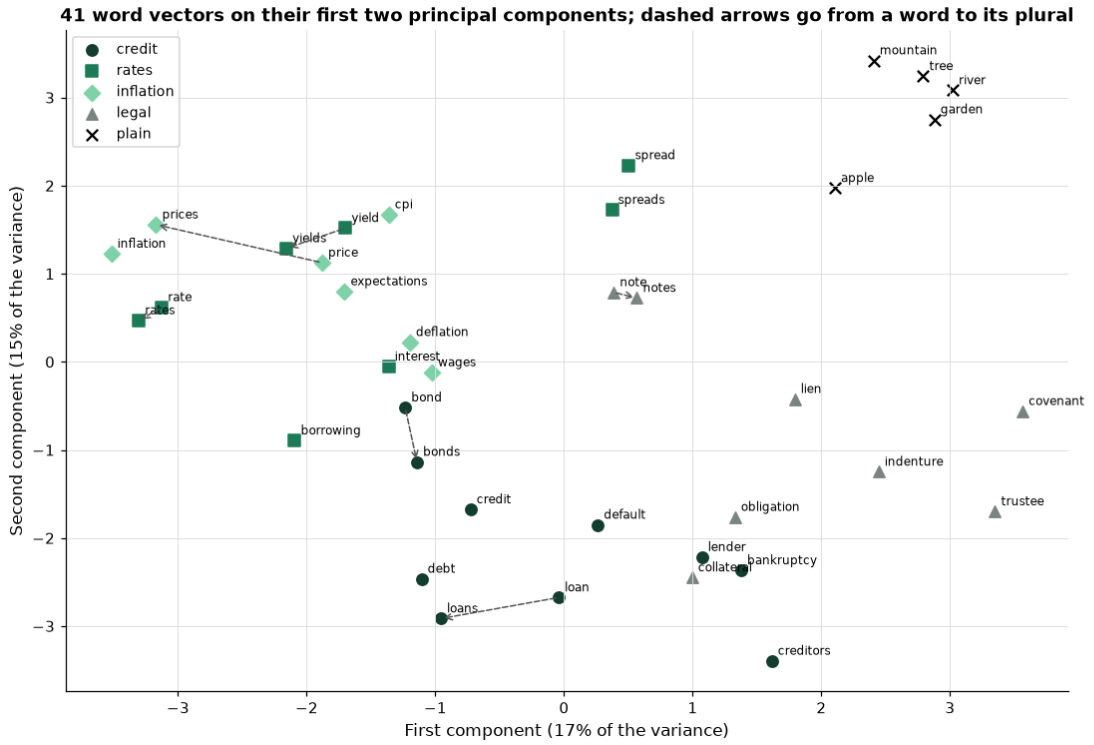
Stop words are a share of 0.4984. A mean has no word order, and every token weighs the same.

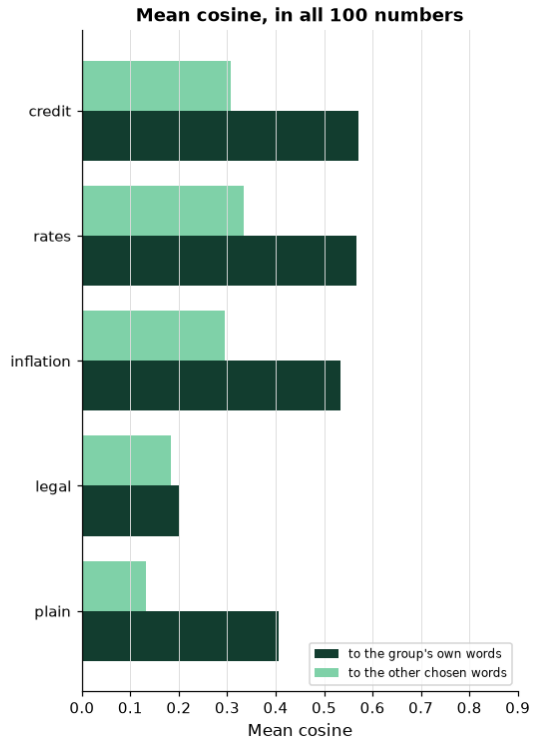
Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 4.12.

How a map of words works

The problem	A word vector has 100 numbers and a page has two axes
PCA	Finds the two directions along which the chosen words differ most
The plot	Each word at its position along them: near in the vectors, near on the page
You control	Which words, and how many components
What comes out	A scatter whose distances are a rough picture, with the variance it keeps

A map of words





31%

of the variance kept

0.208

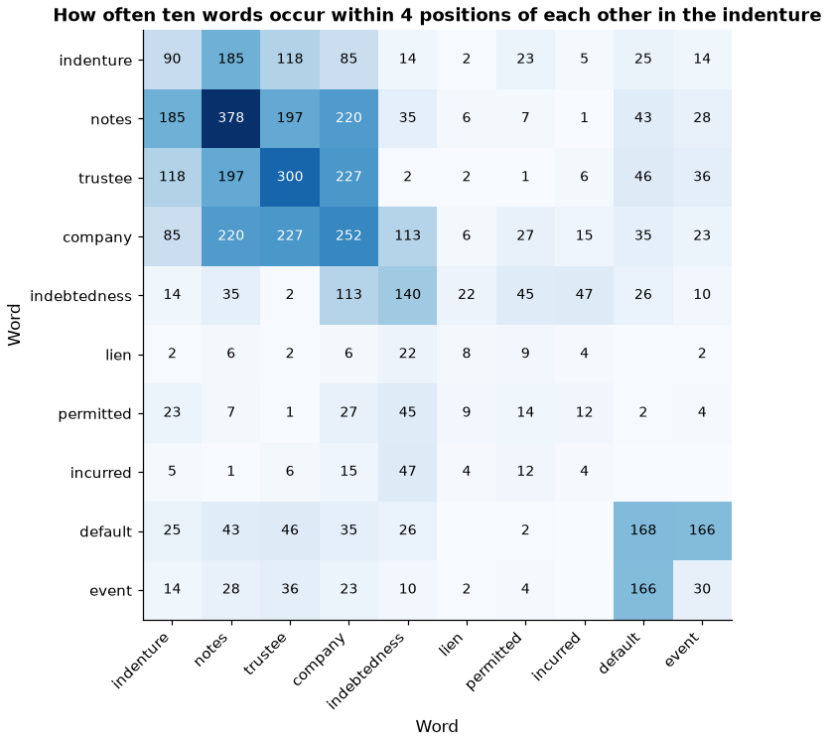
mean cosine, plural offsets

Credit, rates, and inflation words hold together. The legal words, rare in general English, do not.

How word vectors are built

The idea	A word is known by the company it keeps
GloVe	Counts word pairs over a large text and fits vectors whose dot products track the log of the counts
Word2Vec	A small network learns to predict the words around a word; its weights are the vectors
Count-based	Count pairs in a window, keep those above chance (PPMI), squeeze with an SVD
You control	The window, the vocabulary, and the number of dimensions

Counting word pairs



30,525
content tokens

207
vocabulary words

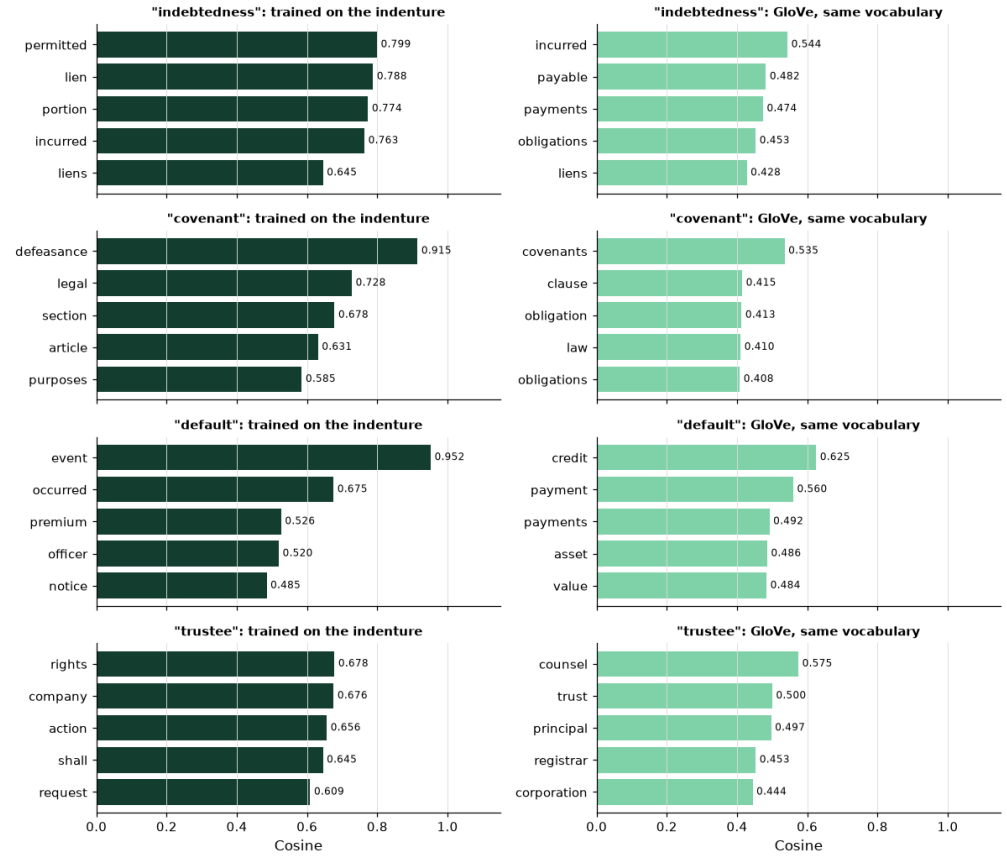
150,116
pairs, window 4

378
largest of the ten

Indebtedness sits beside permitted, lien, and incurred: the words of Section 4.12.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

The indenture's own vectors



indebtedness: permitted, lien, portion, incurred in the indenture's counts; incurred, payable, payments in GloVe.

covenant: defeasance, from Covenant Defeasance. default: event, from Event of Default.

Each set describes the text it was counted from. These vectors rest on 207 words: an illustration, not a tool.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to an 8-K on sec.gov.

Excel side by side

Excel side by side (tested in Excel)

`=SUMPRODUCT(A1:A100,B1:B100)/(SQRT(SUMSQ(A1:A100))*SQRT(SUMSQ(B1:B100)))`	0.8374, within 1e-12
`=SQRT(SUMSQ(A1:A100))`	5.8566
`=AVERAGE(D1:H1)`	0.0353: the mean vector, row 1
`=MATCH("Bonds",J1:J5,0)`	2: capitals ignored
`=XLOOKUP("Bonds",J1:J5,K1:K5)`	1.0565
`=EXACT("Bonds",J2)`	FALSE

No word vectors in Excel: the numbers come from GloVe's file. Python's lookup is exact.

Save the day: read the history

1. Start

.gitignore and mlkit, unchanged

2. textkit

three functions, five tests

3. results.csv

three models on validation

```
git log --oneline --stat -3
Log three models of the 8-K items on the 2024 validation sections
textkit day 4: load_word_vectors, nearest_words, document_vectors, 25 tests
Start the day: .gitignore and Course 4's mlkit, unchanged
```

pytest: 25 passed. The newest commit is at the top.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the lookup never lower-cased: 1,458 of 1,840 tokens used, and a missing share of 0.0000.

Code that runs is not the same as code that is right.

What you can do now

- Load word vectors; compare directions with cosine
- Read nearest words as the training text's usage
- Count what the vectors miss in an indenture
- Use mean vectors as features beside TF-IDF
- Cosine and the mean in Excel; lookups and capitals

Week 1, Day 5 FOMC statement tone over time